

nd here.

Testing

The CustomersDot has different types of tests running:

1. Linting and rubocop jobs

1. Unit tests (specs, these could be of many types, such as controller specs)

1. Integration tests (specs, mocking external calls)

1. Frontend tests

1. E2E integration tests via Watir

We also have a flag VCR that mocks external calls to Zuora by default. We have a daily pipeline that runs at 9AM UTC with the flag set so the API calls hit the Zuora sandbox and we are notified of any failure (due to potential API changes).

Any test failure is notified to sfulfillmentstatus including a link to the pipeline. Pipeline failures will prevent deployments to staging and production.

Security

Access review

Fulfillment Engineering's Engineering Managers and Senior Leadership are responsible for the Fulfillment system (e.g., CustomersDot) access review on a quarterly basis. We use the following guidance to perform access reviews, access for systems will be reviewed based on the job roles and departments. As such, reviewers use their best judgement when evaluating the list of team members who currently have access while evaluating their current roles and department associations.

The following list contains some of the standard departments and teams who should retain access:

Read-only Access

- Sales Team

Write Access

- AppSec Team

- Billing Team

- Fulfillment Sub-department (engineering, product, quality, and other counterparts)

- IT Helpdesk

- Support Team

When in doubt, err on the side of rejecting access as it can be easily restored through another access request.

Deployment

Revenue impacting changes

Fulfillment is unique in Engineering because our changes can directly impact revenue. Changes which impact revenue or are otherwise high risk may require broader scrutiny including input from teams outside of Fulfillment. PM is the DRI on these decisions, taking input from stakeholders including Sales, Enterprise Applications, Marketing, Finance, and Customer Support.

We aspire to be as iterative as possible including even when a change potentially poses high risk.

Below are examples of changes that can be high risk because they can require coordination across many teams:

- Pricing changes
- Billing changes
- Launching a new paid feature
- Deprecating an existing paid feature
- Changes to terms of service and related contract changes
- Changes to the way the consumption of resources is calculated or displayed (e.g., user counts, compute minutes)

Below are changes that can often be low risk:

- Backend enhancements that are not yet used by the frontend
- Frontend and backend Enhancements that are behind a feature flag
- Frontend enhancements that are not behind a feature flag, but are labeled as "beta" or "experimental"

The following processes should be followed for high-risk changes:

- Update feature documentation of the new/updated behavior and share with relevant stakeholders
- Use confidential issues for these changes
- Release changes under a feature flag so this can be enabled on the release date
- Create a rollout issue for the release date and make sure all parties are informed about the process. Example
- Consider enabling this to a subset of customers so changes can be tested iteratively
- Use the regular MR process even if the issue is confidential until confidential MRs are improved (1, 2)

CustomersDot

We use CD (Continuous Deployment) for CustomersDot and an MR goes through the following stages once it gets merged into the staging branch:

mermaid

graph TD;

A(Merged) --> |Green tests| B(Staging);

B --> C[E2E Test on Staging in 40 minutes];

C --> D[Verification]

D --> E(Auto deploy to production in 3 hours);

If something goes wrong at the Verification stage, we could create an issue with the label `production::blocker`, which will prevent deployment to production. The issue cannot be confidential.

When a production blocker is in place and an MR with a fix is needed in order to remove it, the production deployment can be expedited for the MR.
The job to deploy to production needs to be unscheduled to skip the 3 hour delay and triggered manually to deploy right after removing the `production::blocker` label or closing the issue.

For MRs with significant changes, we should consider using feature flags or create an issue with the `production::blocker` label to pause deployment and allow for longer testing.

Feature freeze

The feature freeze for Fulfillment occurs at the same time as the rest of the company, normally around the Friday the milestone ends.

App	Feature freeze ()	Milestone ends
---	-----	-----
GitLab.com	From the Friday the milestone ends to release day	Same as the freeze
Customers/Provision	From the Friday the milestone ends to release day	Same as the freeze

() feature freeze may vary according to the auto-deploy document.

Any issues not merged on the current milestone post feature freeze, will need to be moved to the next one (priority may also change for those).

Production Change Lock (PCL)

There are times when GitLab temporarily halts production changes during certain events such as major global holidays, and other times where GitLab Team Member availability is substantially reduced. More information about PCLs can be found in this [infrastructure handbook page](#).

During these PCLs, most notably at the end of the year, PCLs are managed in CustomersDot by creating an issue using the PCL template. This issue should include a checklist of instructions along with DRIs and target times for adding or removing the `production::blocker` label. Once the PCL has ended, the issue can be closed.

Zuora Blocked Period

Zuora follows a release calendar with blocked periods where Change Requests may not be possible or require additional approvals.

Incident management

On going Fulfillment Platform work will provide greater observability to all of the Fulfillment systems. In the meantime, however, we have some tools like CustomersDot health that post to sfulfillmentstatus in Slack. On occasion, there are reports of systemic critical trouble like a liveness probe check failure, invalid SSL certificate, or other application errors. Look to the following sub-sections below for ways that you can contribute to error resolution.

See this handbook page for more information on GitLab Monitoring and Incident Management.

Escalation process for incidents or outages

Temporarily, while pagerslack or pagerduty is not adopted in Fulfillment, the following process is in place:

1. When an outage occurs, the SRE on-call is notified automatically. Incidents can also be reported manually.
1. Simultaneously to an outage, sfulfillmentstatus on Slack is notified and James Lopez and Vitaly Slobodin are paged on the phone
1. When required, the SRE on-call or incident reporter can ping @fulfillment-engineering on Slack to notify and get help from the team

Escalation process for urgent issues

In most cases an MR should follow the standard process of review, maintainer review, merge, and deployment as outlined above. When production is broken:

1. First determine whether the Rapid Engineering Response process should be followed. This will depend on availability and how critical the situation is.
1. The three hour wait between Staging and Production auto deploy can be bypassed with a manual deployment by a Maintainer.
1. When there are no project maintainers available (CustomersDot) an additional GitLab Team Member with Maintainer access can be asked to assist.

In these cases please ensure:

1. There is an issue describing the reasons for the escalation, as per the Rapid Engineering Response. Consider '@'mentioning the relevant Growth/Fulfillment teams.
1. The change is announced on the sfulfillment channel.

Investigation

- You can visit our health-check instance here. You can find the login credentials in the Subscription portal vault in 1Password. You will see services for both CustomersDot production and staging.
- Exceptions for CustomersDot are captured in Sentry.
- You can query logs in Kibana / Elasticsearch. Use this link to read more on using kibana at GitLab)
- Grafana has some triage dashboards that are generally applicable for GitLab, but do not contain specific observability metrics for Fulfillment systems until the InfraDev work is complete.
- Blackbox probes reporting production availability alerts are reported to production in Slack

Declaring incidents

Raising an incident is always an option if you need immediate attention on a blocking problem.

Examples of blocking problems include:

- Expired certificate causing interrupted service with staging or test environments

Critical problems like a production outage should be raised quickly. You can check incidentmanagement before raising an incident.

- CustomersDot outage
- Failed CustomersDot deploy

Grafana dashboards

These dashboards are designed to give an insight, to everyone working in a feature category, into how our code operates at GitLab.com scale.

- Provision
- Utilization

Customer escalations

If there is a licensing issue (from support or sales) that requires escalation to product management or engineering, please create an issue via the process documented on the support handbook page under Assistance with License Issue. Please do this instead of escalating in Slack or other methods.

Fulfillment leaders in product management and engineering will subscribe to the License Issue High ARR label so they can be aware of them when they are created. This can be done in via a label search and then clicking on 'subscribe' for the `License Issue High ARR` label.

Retrospectives

Monthly retrospectives

After the 8th, the Fulfillment team conducts an asynchronous retrospective. You can find current and past retrospectives for Fulfillment in <https://gitlab.com/gl-retrospectives/fulfillment/issues/>.

Retrospective Contributions

Everyone is encouraged to participate in asynchronous retrospective issues. This is a safe space for team member feedback. You are welcome, however, to bring your comments and concerns to the synchronous retrospective discussion or share them with your manager or other department leadership as well. We follow this guidance to produce an efficient and safe retrospective.

Some things to keep in mind:

- Items shared do not have to be monumental successes or failures. Consider including even small items in the "what went well" and "what didn't go so well" sections of the retrospective issue.
- Use a team specific thread to share items that may feel less relevant to the entire department.
- Pull from the "wins" or "challenges" team sync agenda items for inspiration.
- Find the limits of your computer, and keep the current retrospective issue open throughout the milestone.

Asynchronous Retrospective Synchronous Discussion

Approximately one week after the commentary period closes (on the 26th of the following month) for the async retrospective, we host a synchronous meeting to discuss next steps, action items, and improvements to our processes. This meeting can be facilitated by EMs and ICs alike. We attempt to be timezone inclusive by scheduling the meeting on a rotation between times that are APAC, AMER, and EMEA friendly. This meeting is recorded and shared for the benefit of everyone in Fulfillment.

Iteration retrospectives

There are several resources at GitLab for improving our iteration capability:

- engineering iteration documentation
- product iteration documentation
- self-guided iteration training issue
- iteration office hours
- milestone retrospectives
- iteration retrospectives

Agile software development practices uses the phrase Iteration Retrospective to describe what we perform during our Milestone Retrospective. While the Milestone Retrospective is essential in understanding how we can improve, it does have a broad focus across multiple teams. It's a safe space where everyone can contribute through reflection on what we can continue, what we can correct, and what we can improve.

An Iteration Retrospective, in the GitLab values sense that we're using here, is more focused on a specific example (or few examples) of a successful or unsuccessful iteration attempt. That is, an Issue, Epic, or merge request that we can examine deeply to understand what we can do differently in the future to be more iterative and efficient. These retrospectives will provide a growth opportunity to work on developing an iterative mindset - build it, then build it better.

Each group should perform their own Iteration Retrospective, but share as transparently as possible to help fellow teammates and other teams learn from your investigation.

Frequency

Iteration Retrospectives should be conducted quarterly.

Preparation

Engineering Manager(s) or Other team members should review current or previous milestones to find a good, challenging candidate issue for the Iteration Retrospective. Choose one (or two if related) from the suggestions, then create a retrospective issue.

Review these rules for an effective retrospective.

Participation

Everyone can and should contribute asynchronously to the discussion. Present your ideas on alternative methods or mechanisms to break down the problem stated in chosen issue for review. If the problem breakdown was fine, suggestions about process or other refinements are also welcome.

Don't belabor the point, participation should be timeboxed to 1 week. A 1 week due date quick action is available in the template below.

Consider handbook updates, process changes, and bubbling up information from your conclusions after this retrospective.

Template (optional)

markdown

Iteration is one of six GitLab Values, but also really difficult. By focusing how we have iterated well in the past and how we have not will help us iterate faster. Please contribute the following Iteration Retrospective.

Summary

<!--

Include a brief summary of the issue. Consider including key outcomes, significant changes, and impact.

-->

Summary of the Issue

Details

<!--

Complete the following details when creating the issue.

-->

- Did the team meet the iteration goals? Why or why not?
- How many MRs were created to complete the Iteration?
- What were the Days to Merge for the MRs related to the this issue?
- Are there successes or opportunities for improvement with respect to collaboration with peers or stable counterparts?
- Could the original issue have been broken down into smaller components?
- Could the MR(s) have been broken down into smaller components?

Tasks

Read

- Iteration Value
- Engineering Iteration
- Why iteration helps increase the merge request rate

Watch

- Interview about iteration in engineering with Christopher and Sid
- Iteration Office Hours with CEO

Contribute

In the threads below consider the following:

- Why was it (un)successful? If successful, how did it meet the definition of an MVC?
- How did the example not meet the definition of an MVC? In what ways could you have iterated differently? List specific examples.
- Identify areas of improvement for the team that can be incorporated into our processes and workflows.

Follow these rules for an effective retrospective.

Check your name off the completed list when all tasks are complete.

Completed

<!--

When creating this issue replace the person placeholders with teammembers and select stable counterparts as you see fit.

-->

- [] Person
- [] Person
- [] Person
- [] Person

Next Steps

- [] Engineering Manager to update your team's handbook page with ideas and improvements you've incorporated to improve iteration.

Thread Prompts

<!--

Use these prompts or similar to start conversation threads.

-->

1. What aspects of the Issue/Epic and resulting MR(s) were very good/bad examples of Iteration

and why?

1. Does anyone have alternative ideas for how this work could have been broken down?
1. Is there anything that we can change in our processes or the way we work that could improve our iteration as a team?

/due in 1 week

/label ~"Iteration Retrospective"

Project Cadence

This table lists recurring activities that are part of our Project Management Process. Having a well defined cadence makes it simple for our team to understand what activities are happening and when they happen. The listed activities closely follows our company cadence. Use this spreadsheet to edit this markdown table.

Activity	Cadence	Type	Teams Involved
GitLab.com Deployments	Continuously	Async	Delivery Engineering
CustomersDot Deployments	Continuously	Async	Fulfillment Engineering
Issue Updates	Weekly	Async	Fulfillment Section
Fulfillment Weekly Updates	Weekly	Async	Fulfillment PMs, EMs, QEMs, UXMs
Fulfillment Group Syncs	Weekly	Sync	Fulfillment Utilization, Provision, and InfraDev Groups
Fulfillment PM Syncs	Weekly	Sync	Fulfillment PMs
Fulfillment, Growth, AI Assisted EM Syncs	Weekly	Sync	Fulfillment, Growth, AI Assisted EMs
Fulfillment PM / EM / QEM / UXM Syncs	Weekly	Sync	Fulfillment PMs, EMs, QEMs, UXMs
OKR Check-Ins	Weekly	Async	Fulfillment PMs, EMs, QEMs, UXMs
Monthly Self-Managed Releases	Monthly	Async	Delivery Engineering
Monthly Release Posts	Monthly	Async	Product and Marketing Functions
Milestone Planning	Monthly	Async	Fulfillment Utilization,, Provision, and InfraDev Groups
Roadmap Planning	Monthly	Sync	Fulfillment PMs, EMs, QEMs, UXMs
Retrospective Issue	Monthly	Async	Fulfillment Section

Retrospective Discussion	Monthly	
Sync Fulfillment Section		
Direction Review		
Quarterly Async Fulfillment PMs, Product Leadership		
OKR Planning		
Quarterly Async Fulfillment PMs, EMs, QEMs, UXMs, Staff Engineers		
Board Meetings		
Quarterly Async Fulfillment PMs, E-Group, Board		

Working Groups and Cross-Functional Initiatives

This table lists recurring activities that are part of working groups and cross-functional initiatives. Use this spreadsheet to edit this markdown table.

Activity	Cadence	Type	Teams Involved
-----	-----	-----	-----
-----	-----	-----	-----
Purchasing Reliability Working Group	Weekly	Sync	Fulfillment Engineering, Infrastructure, IT, CEO, Office of the CEO
GTM Product Usage Data Working Group	Weekly	Sync	Fulfillment PMs, Analytics Instrumentation, Data, Customer Success, Sales
GitLab Order to Cash Technical Fusion Team	Weekly	Sync	Fulfillment PMs, Fulfillment Engineering, EntApps, Sales Systems
Data & Analytics Program for R&D Teams	Every 2 Weeks	Sync	Fulfillment PMs, Analytics Instrumentation, Growth, Data
Cloud Licensing Executive Status Update	Every 2 Weeks	Sync	Cloud Licensing Program Manager, E-Group
Ramps / Orders Bi-Weekly	Every 2 Weeks	Sync	Provision, EntApps, Channel Ops, Field Ops, Support, Finance, Legal
Product ARR Drivers Sync	Monthly	Sync	Customer Success, Sales, Product Leadership
Distributor E-Marketplace	Monthly	Sync	Provision, EntApps, Channel Ops, Field Ops, Finance, Legal

Diversity, Inclusion, and Belonging

See the Fulfillment Section DIB page for more information about our DIB initiatives, including Fulfillment social events.

Performance indicators

See the Fulfillment Section Performance Indicators as well as the Centralized Engineering Dashboards.

Maintainer ratios

CustomersDot

Maintainer Ratio Dashboard

We attempt to have an ideal Engineer to Maintainer Ratio of 1:1 for both backend and frontend effort for our primary application, CustomersDot. The active list of maintainers can be found on the Engineering Projects page.

Some considerations that differ from other maintainer ratios:

1. Fulfillment maintainer ratios are Maintainers vs Reviewers
1. GitLab maintainer ratios are Maintainers vs Maintainers + Reviewers
1. GitLab maintainer ratios do not count full-stack engineers properly (Fulfillment one does)

Knowledge sharing

- Generating test licenses, adding them to GitLab, and then finding and removing them from the GitLab database
 - Document with notes
 - Recording
- New Integration with Sentry
 - Document with notes
 - Recording
- Intro to FE GraphQL at GitLab with Vue Apollo
 - Slides
 - Recording
- CustomersDot Resource Video Library
- Zuora University Select Topics
- Confidential Issue Git Pre-push Hook to prevent any accidental push to any non-GitLab Security remote whenever a developer works on a confidential issue. Reference: How to customize Git Hooks.
- Pair programming in Fulfillment
 - A great way to knowledge share between engineers in the section is by working together on a particular issue. Learn more about the benefits and how to request a pairing session in the doc.

Managers and direct reports

This table lists recurring activities between managers and their reports. Most of these activities are also listed on our Leadership and People Group pages. Use this spreadsheet to edit this markdown table.

Activity			
Cadence	Type	People Involved	

1-1 Meetings			Weekly Sync
Manager, Report			
Career Development Conversations	Quarterly	Sync	Manager, Report
Skip-Levels Meetings			Quarterly Sync

Manager's Manager, Report	
Promotion Planning	Annually Sync Manager, Report,
Function Leadership, PeopleOps	
Talent Assessment	Annually Sync Manager,
Report, Function Leadership, PeopleOps	
Annual Compensation Reviews	Annually Sync Manager, Report, Function
Leadership, PeopleOps	

Google groups

Google groups can be used for easily sending calendar invites to members of the group.

Group name

fulfillment-engineering

Communicating on Slack

List of channels

Channel Name Purpose
----- -----
sfulfillment Used for asking questions regarding the product, engineering, and Fulfillment processes.
sfulfillmentfyi Used for announcements related to the Fulfillment sub-department.
sfulfillmentengineering Used by the Fulfillment sub-department engineering team for internal communication and resolving internal engineering-related queries. Requests for pair programming sessions from other Fulfillment engineers can be made here.
sfulfillmentdaily Used to share daily standup updates.
sfulfillmentstatus CustomersDot health monitoring channel.

Guidelines to ask questions in sfulfillment slack channel

1. The sfulfillment slack channel can be used to ask all product, engineering, and Fulfillment processes related questions.
1. When your question has been answered, please react with the · (:whitecheckmark:) Slack emoji to the original question so that team members know that it has been answered and they do not need to look at it any longer.
1. For urgent support issues, please follow the STAR escalation strategy.
1. For internal support requests regarding licensing, subscriptions, trials, grace period extensions, or any requests involving a customer, please fill out the internal request form.
1. For general licensing and subscription questions, check out the supportlicensing-subscription Slack channel.

Common links

- Product Vision
- Fulfillment section
- All open Fulfillment epics
- Issue Tracker

- Slack channel sfulfillment
- Daily standup Slack channel sfulfillmentdaily
- Fulfillment announcements Slack channel sfulfillmentfyi
- Team calendar
- Fulfillment technical debt status (to be migrated to Tableau)

SECTION: engineering/development/fulfillment/dib.md

Fulfillment Social

We like to stay connected and up-to-date with our team outside of our daily work and foster an inclusive environment.

Activity	Cadence	Type
sfulfillmentsocial Slack	Continuously	Async
Fulfillment Weekly Social	Weekly	Sync
Fulfillment Team Days	Quarterly	Sync

Slack channel

We use the private sfulfillmentsocial channel to chat and socialize.

Social calls

We hold optional synchronous social meetings weekly where we chat about anything outside work. These meetings happen each Wednesday on three time slots, 9:00 UTC (EMEA) 16:00 UTC (AMER) 22:30 (APAC)
Check the Fulfillment Google calendar for more information.

Team days

Every quarter, we set aside our work and host a virtual get-together to socialize with our fellow teammates. Our upcoming and past team days are listed in the Fulfillment Meta Project using the Team Day label.

Onboarding

We have Fulfillment onboarding templates (BE/FE engineer, UX engineer), Software Engineer in Test.

As part of onboarding, you will be assigned an onboarding buddy that will make you feel welcomed as well as schedule a number of coffee chats to get to know other Fulfillment team members.

MR DIB and collaboration metrics

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/FulfillmentEngineeringMRDIBandcollaborationmetrics/AppreciationRate" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/FulfillmentEngineeringMRDIBandcollaborationmetrics/Celebrationrate" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/FulfillmentEngineeringMRDIBandcollaborationmetrics/Sympathyrate" >}}
{{< /tableau >}}
```

SECTION: engineering/development/fulfillment/fulfillment-platform.md

Vision

More details about the product vision for Fulfillment Platform, can be found on one of the Fulfillment Platform direction pages.

For a wider picture of the product vision for the Fulfillment section, please see the Fulfillment direction page.

The Platform group manages the Platform category.

Team members

```
{{< team-by-manager-role role="Manager, Fulfillment:Fulfillment Platform" team="[,&]Fulfillment:Fulfillment Platform" >}}
```

Stable counterparts

```
{{< engineering/stable-counterparts manager="Manager, Fulfillment:Fulfillment Platform" role="(?)Fulfillment Platform|(?)Fulfillment section" >}}
```

Performance indicators

```
{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="fulfillment platform" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-
```

```

west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}
  {{< tableau/filters "GROUPLABEL"="fulfillment platform" >}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-
west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}
  {{< tableau/filters "GROUPNAME"="fulfillment platform" >}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-west-
2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard"
>}}
  {{< tableau/filters "GROUPLABEL"="fulfillment platform" >}}
{{< /tableau >}}

```

You can find the centralized engineering metrics at this page.

For Product Performance Indicators please visit the internal handbook.

OKRs

The Engineering Manager will report on the progress of projects rolling up to CEO OKRs every week. Current OKRs: FY25-Q1

Architecture review sync meeting

We want to use our Architecture Review meeting to invite other people from the Fulfillment Section. We aim to increase transparency, collaboration and knowledge sharing in the Fulfillment Section. Further we can use it as internal user research for the group.

Meeting focus

- Upcoming projects that need cross-group collaboration (e.g. CustomersDot BillingAccount can have multiple CustomersDot Users).
- Intake and proposals to enable other Fulfillment groups to be more efficient.
- Architectural reviews and suggestions (e.g. spikes from other teams) that impact the CDot architecture.
- Topics related to our Zuora integration.
- Topics related to any Fulfillment product category.

Meeting routine and agenda

The weekly meeting can be seen on the Fulfillment calendar. Everyone is welcome to attend.

Every meeting has one or more discussion topics. To be inclusive and mindful of team members in all timezones, please consider the following:

- Topics should be added to the agenda well in advance, and at least one day before the meeting.
- Agenda with discussion topics will be shared in sfulfillmentfyi slack channel one day before

the meeting.

- Topics can be reviewed async before the meeting, so questions and answers can be added to the agenda document.
- If there are no discussion topics in the agenda one day before the meeting, it will be canceled.

Upcoming topics

Upcoming and past topics are documented in the internal Architecture review meeting topics agenda doc.

How to bring a topic to the meeting

We want everybody to contribute to the meeting. If you have a topic that you want to bring up in the Architecture Review meetings, add it to this list of topics in the internal agenda doc and we will schedule it.

We ask you to submit the agenda topic at least one day in advance of the meeting.

SECTION: engineering/development/fulfillment/provision.md

Vision

For more details about the product vision for Fulfillment, see our Fulfillment page.

The Provision group manages the Provision category.

Team members

{{% team-by-departments "Fulfillment:Provision Team" %}}

Stable counterparts

{{< engineering/stable-counterparts manager-role="Fullstack Engineering Manager(.)Fulfillment:Provision" role="Fulfillment:Provision" >}}

Project management process

The Provision project management process inherits from the general Fulfillment project management process and adds a few additional elements specific to the work that the Provision team does.

Planning

The milestone planning process is built around planning issues for the milestone, these issue outlines what issues will be worked on in the upcoming milestone, how those issues are prioritized, commentary on how that fits with over arching goals, and how the proposed work fits in the capacity for the next month. The Provision team use this template for it's planning

issues.

Planning Timeline

The team Engineering Manager and their Product Management counterpart are responsible for implementing the milestone planning process. To complete all of the preparation work needed for each milestone the Provision team uses a schedule of:

- 1st of the month create new planning issue, ensure team member monitoring availability calendar is filled in for the upcoming month
- 9th of the month request engineer input and refinement of candidate issues
- 13th of the month finalized PTS and Sentry monitoring schedule
- 14th of the month final capacity planning for refined issues
- 18th of the month milestone starts

Refinement

The refinement process aims to ensure that all planned work can be completed within the scheduled milestone. During the refinement phase issues are distributed to engineers to answer three questions:

- Are there any open questions that prevent this issue from being marked as workflow::ready for development?
- Are there any design problems that could lead to predictable blockers coming up while implementing this issue?
- What weight estimate would you give for this issue?

Clearing foreseeable blockers helps to ensure that all scheduled work can complete without stalling. If there are issues that can not be completed due to design considerations those should be pushed back to a future milestone to ensure capacity is directed to issues that can be delivered in the upcoming milestone. Having accurate estimates of the work needed for planned issues is important to ensure that all planned work can be completed accounting for team capacity.

Capacity Planning

The Provision team is primarily a development team, however given the nature of the team's work there are additional operational concerns that routinely take team member's time away from purely development tasks. The main recurring tasks that take team member time are:

- Monitoring the Provision Tracking System
- Monitoring Sentry for integration related errors
- Triaging licensing issues impacting customers

To account for the time consumed by the on going operational processes for each milestone a issue with a weight of 3 is created for Sentry monitoring and assigned to the engineer that is DRI for that process for milestone. For Provision tracking monitoring 4 issues with a weight of 1 each are created and assigned to engineers based on the Provision Tracking Monitoring schedule for the milestone.

The target weight for issues closed during a milestone is 0.5 (Total number of working days accounting for time off). There is an open issue discussing ways to move to planning capacity to a per engineer granularity.

Change Management

In the spirit of Iteration and Collaboration these processes are a work in process. Suggestions on improvements or new processes to try can be proposed under this epic.

Historical capacity

Points of weight delivered by the team on the last milestones. This allows for more accurate estimation of what we can deliver in future milestones. Full chart to be migrated.

Performance Indicators

```
{{< engineering/child-dashboards filters="Provision" >}}
```

OKRs

The Engineering Manager will report on the progress of OKRs every two weeks. Current OKRs: FY22-Q1

SECTION: engineering/development/fulfillment/seat-management.md

Overview

Placeholder page for Seat Management Team

SECTION: engineering/development/fulfillment/subscription-management.md

Overview

Placeholder page for Subscription Management

SECTION: engineering/development/fulfillment/utilization/index.md

Overview

The Utilization Team often works at the interface between GitLab Core and Fulfillment applications. This includes components in the Utilization category like consumables management (storage, compute, seats, etc.), usage reporting, and usage notifications. Our customers use GitLab SaaS, GitLab self-managed, and internal tooling.

Vision

For more details about the product vision for Fulfillment, see our Fulfillment page.

The Utilization group manages the Utilization category.

Team members

```
{{% team-by-departments "Fulfillment:Utilization Team" %}}
```

Stable counterparts

```
{{< engineering/stable-counterparts  
role="Fulfillment:Utilization|Security(.)Fulfillment|Director(.)Fulfillment" manager-  
role="Fullstack Engineering Manager, Fulfillment:Utilization" >}}
```

How we work

Sync Time is Valuable

We try to work async as much as possible. However, there are occasions where synchronous communication might be better suited for the task. Our Weekly Team Sync meeting is one of those occasions where we can celebrate wins, collaborate quickly over a number of issues, and generally have some face-time in an all-remote environment.

Crucially, we use this high bandwidth communication to review issues that need some extra discussion and input from the team to ensure that issues can be estimated and work developed with minimal back and forth to establish the nature of the request.

Times and Timezones

The Utilization group is spread across at least 4 different timezones.

The Utilization Team meets via Zoom on Tuesdays each week at 9:00 AM US Eastern Time (6:00 AM PT / 2:00 PM UTC) to accommodate teammates in their timezones.

Recording Meetings

Using the REC] meeting prefix, the meeting is automatically uploaded into the [GitLab Videos Recorded folder in Google Drive on an hourly basis via a scheduled pipeline. All teammates are set as alternate hosts and should be able to record the meeting if the Engineering Manager is not present. The recording link will be placed into the agenda after the recording is available.

We don't normally publish our team sync meetings to the GitLab Unfiltered YouTube channel because we often talk about confidential issues related to product pricing and other internal initiatives. We would rather have a single repository of meeting recordings in Zoom or Google Drive rather than risk of unsafe information disclosure.

Meeting Preparation

All team members are encouraged to add topics to the weekly agenda async. In the unlikely event that the agenda is empty, we'll cancel the meeting.

Async Updates

Engineers

Engineers are responsible for providing async issue updates on active, assigned issues when progress is made. Following the template and guidance for async updates for the entire Fulfillment Sub-department, updates should be made at least weekly. These updates help keep collaborators, stakeholders, and community contributors informed on the progress of an issue.

In addition to weekly issue updates, we ask that engineering DRIs also provide a weekly status update for the bodies of work where they're focussed.

- When: We aim to provide these updates every Tuesday
- Who: Anyone who is listed as a DRI against our current milestone priority projects
- What: The updates should include accomplishments, any blockers and a sense of how the work is progressing against the timeline. As an added bonus, a link to a demo that has been posted in the sfulfillmentdemo Slack channel would also be beneficial.
- Where: Please provide them in the comments of the primary/parent epic of your work and tag Alex and James when you do so.

An example template to use in these updates looks like:

markdown

BE Weekly Update YYYY-MM-DD

Timeline Confidence: :thumbsup:

% Complete: X% (Y weight closed / Z total weight)

Blockers: None

Accomplishments:

3 MRs created

1 issue closed

cc/ @alexmartin @csouthard

Initially we are using the following formula for % Complete, rounded down to the nearest multiple of 5:

markdown

$$(\text{issue weight closed} / \text{total issue weight}) \times 100$$

Engineering Manager

Milestones

The Engineering Manager will report before the end of each week on milestone progress in the

current milestone planning issue on the following topics:

markdown

Total Weight Closed XX

Total Weight Open XX (XX in dev)

Deliverable Weight Closed XX

Deliverable Weight Open X (XX in dev; X blocked)

Blocked Issues X issue(s) (X weight) link, description

OKRs

The Engineering Manager will report on the progress of OKRs every two weeks as a comment in relevant work items found in the Objectives and Key Results project.

Current OKRs: Work Items Labeled with Utilization

Taking Time Off (PTO)

It is important to take time off so that you can rest, reset, avoid burnout, take care of loved ones, etc. You are encouraged to take time for yourself or your family following our operating principle of Family and Friends First, work second. Guidance is to communicate your PTO intention with your manager with twice the number of days that you are taking off.

When going out of office, please be sure to clearly communicate your availability with other people. The following steps are required when submitting a PTO notification.

1. In Time Off by Deel, select a role as your backup during your PTO. Please assign the team slack channel gutilization as your backup to help distribute the workload. Consider if your current work in progress requires a substitute DRI and assign a single person for those specific issues.
2. Add the Fulfillment Shared Calendar to your Time Off by Deel settings so your PTO events are visible to everyone in the team. The calendar ID is:
gitlab.com7199q584haas4tgeuk9qnd48nc@group.calendar.google.com Read more about PTO in the handbook.
3. Update your GitLab.com status with your out of office dates by clicking on your profile picture and selecting "Edit Status." For Example: OOO Back on yyyy-mm-dd Adding OOO to your status message will keep you from appearing in the reviewer roulette.
4. If the planned PTO is more than 3 days, create a coverage issue to indicate who's helping where and how we all can support each other. If you're unsure if a coverage issue is needed for your planned PTO, please discuss with your engineering manager.

Milestone Planning

Key dates

| Date | Event |

| ----- | ----- |

| The Monday of the week the milestone ends | PM creates a Planning Issue and pings the EM(s) in the Planning Issue for review & preliminary weighting.

 EM and PM calculate capacity, add to Planning Issue. |

| Monday to Friday of the week the milestone ends | EM & ICs add weights to issues in the backend and frontend build boards. |

| The Friday the milestone ends | PM adds ~Deliverable labels to issues. |

| The Friday the milestone ends | Last day of milestone

 PM adjusts current and upcoming issues to reflect slippage from the current milestone. ~Deliverable labels are adjusted as necessary. |

| Team Sync Closest to the Next Milestone | PM reviews the upcoming milestone plan with the team. |

| The third Thursday of the month | Release |

How We Prioritize Issues

We have cross-functional prioritization process that is aligned with our prioritization framework.

Responsibilities

- The Product Manager will prioritize type::feature issues
- The Engineering Manager will prioritize type::maintenance issues
- The Software Engineer in Test will prioritize type::bug issues
- The Product Designer will prioritize bug::ux issues (System Usability Scale (SUS))

Mechanics

- The team reviews this dashboard which shows the distribution of MRs that are bugs, maintenance, and features to ensure the team's efforts are properly aligned to our target prioritization ratio (60% features / 30% maintenance / 10% bugs).
- The team uses this prioritization board to order our top features, bugs, maintenance, and System Usability Scale (SUS) impacting issues.
- For a quick view and controls on bugs by priority, this board can be used. More detailed information on bugs, including backlog and open rate, both of which are factors in determining their desired bug percentage for the upcoming milestone, is shown in this dashboard (select Utilization from filters).
- The team collaborates on a prioritization issue that documents and discusses our current prioritization scheme.

Planning Issues

- Current Planning Issue
- Archive 13.8 - present

How Engineering Refines Issues

Before work can begin on most issues, a preliminary investigation is often required to provide an estimate and breakdown of the work as described. If the scope of work for a given issue spans several disciplines (docs, design, frontend, backend, etc.) and involves significant complexity across those disciplines, consider creating separate issues for each discipline.

Engineers are expected to allocate approximately 4 hours each milestone to evaluate, refine, and weigh issues assigned to them. Refining an issue ensures the problem statement is clear enough and designs are available (if applicable) to provide a rough effort sizing estimate; the intention is not to provide solution validation during refinement.

If any issue requires any additional ~frontend, ~backend, ~Quality, ~UX, or ~documentation reviews, they are assigned to the respective individual(s) to collaborate on the issue where they can validate assumptions and/or provide missing detail necessary to provide an estimate.

Checklist for Refining Issues

1. Does the issue have a problem statement in the description?
1. Does the issue have the expected behavior described well enough for anyone to understand?
1. Does the issue explicitly define who the stakeholders are (e.g. BE, FE, PM, UX and/or Tech Writer)? In case the stakeholders aren't very clear (especially when it's not FE nor BE), mention both the PM and EM when you ask for clarification?
1. Does the issue have a proposal in the description?
 - If so:
 1. Does the proposal address the problem statement?
 1. Are there any unintended side effects of the implementation?
 - If not:
 1. Create a proposal during the estimation steps or summarize the prevailing proposal.
1. Does the issue have proper labeling matching the job to be done? (e.g. bug, feature, performance)

Anyone on the team can contribute to answering the questions in this checklist, but the final decisions are up to the PM and EM.

Updated Refinement/Estimation Template

markdown

Refinement / Weighting

<!--

Ready for development means replying yes to the following questions:

- Does the issue have a problem statement in the description?
- Does the issue have the expected behavior described well enough for anyone to understand?
- Does the issue explicitly define who the stakeholders are (e.g. BE, FE, PM, UX and/or Tech Writer)?
- Does the issue have a proposal in the description?
- Does the issue have proper labeling matching the job to be done? (e.g. bug, feature, performance)

- Is this issue sufficiently small enough? If not, break it into smaller issues
- Is it assigned to the correct domain (e.g. frontend, backend)? If not, break it into two issues for the respective domains
- Is the issue clear and easy to understand? If not, try asking further clarification questions and update the description once they are received
- Has the issue been reviewed for overlap with other issues and addressed as necessary?
- Are the weights of the issues updated based on team input and knowledge?

If more than 2 MRs are needed, consider adding a table like the following to the description (e.g. under Implementation plan).

```
| Description | MR |
|-|-|
|
```

It will help track the status.

-->

- ☐ Ready for development
- ☐ Weight is assigned
- ☐ Number of MRs listed
- ☐ Needs testing considerations
- ☐ Needs documentation updates
- ☐ Reviewed other milestone issues for possible overlap with current one and mark them as "linked", "blocks", or "is blocked by"

Reasoning:

<!--

Add some initial thoughts on how you might break down this issue. A bulleted list is fine.

This will likely require the code changes similar to the following:

- replace the hex driver with a sonic screwdriver
- rewrite backups to magnetic tape
- send up semaphore flags to warn others

Links to previous examples. Discussions on prior art. Notice examples of the simplicity/complexity in the proposed designs.

-->

Steps for Refining and Weighting Issues

1. By the 10th of the month, engineers will be assigned to milestone candidate issues by either EM or PM. These issues should have needs weight and workflow::planning breakdown labels. This board tracks those assignments.

1. For the next 5 days, often between the 10th and 14th, engineers should work to provide estimates, refine, and breakdown issues.

1. Use the checklist above to refine issues.

1. Split an issue out into components. For example, if backend work needs to be completed before frontend work can begin. See below for more information.
1. Update associated issues to be linked, marking as blocked where appropriate.
1. Add a weight based on our definition (see below)
1. Update the ~workflow:: label to the appropriate status, e.g.
 - ~"workflow::design" if further design refinement is needed, and let the designer know.
 - ~"workflow::ready for development" when refinement is completed and a weight has been applied, signaling that it's ready for implementation and the issue can now be prioritized.
 - ~"workflow::planning breakdown" if extensive investigation and/or research is needed, the status does not move, and the PM and EMs should be informed. An issue remains in this workflow status while collaborating with others to clarify minor aspects of the issue.
1. Unassign themselves from the issue when they are done refining and weighing the issue.

Splitting issues

If an issue has several components (e.g. ~frontend, ~backend, or ~documentation) it may be useful to split it up into separate implementation issues. The original issue should hold all the discussion around the feature, with separate implementation issues being used across disciplines to track the work done. Doing this provides several benefits:

- There's only one DRI per issue.
- Workflow labels and health statuses are more relevant.
- We can weight issues with more accuracy.
- We can mark one implementation as a blocker for another.
- It's easier to see what work can be picked up by each functional group.
- We can schedule work across multiple milestones.

Re-evaluating issues

Sometimes a long passage of time might elapse between the initial refinement and an engineer picking up the issue to work on it. With the frequency with which the codebase can be updated, this can lead to the implementation plan or the weight being inaccurate and in some cases the issue might not even be required anymore.

If that passage of time is significant, e.g. 1 year, we should re-evaluate the issue to ensure it is still relevant and accurate by performing the refinement process, using the Refinement/Estimation Template above.

Maintenance Issues

Much like Martin Fowler's technical debt quadrant, we'll repurpose the Eisenhower Method to help guide our thinking as we categorize and prioritize our maintenance issues. We use a two-axis framework, urgency and importance, to prioritize maintenance issues (refactoring, intentional or unintentional technical debt, etc). It's a framework for decision making.

urgency: "Will we need it soon?"

Range: not-urgent or urgent

We can also quantify urgency in a number of ways. One method is based on customer impact. For

instance, we can use the number of errors(5xx's code errors; user facing exceptions), error-budget(higher impact leads to higher urgency), or the number of customers impacted by the issue. Other non-customer based urgency could include the timeline for driven changes such as package or gem upgrades.

Defining questions:

1. "What will happen if we don't fix this issue in the next one milestone?"
1. "Does the business need still exist? Is this still actively being used by our customers?"

importance: "How bad is it?"

Range: not-important or important

Importance can be more subjective. From the your perspective, the relative importance will be filtered through the lens of your role or your discipline.

There are some categories that we can agree are objectively important. These include: defects, system performance (REST and GraphQL APIs; database transaction efficiency), and lack of testing.

Defining questions:

1. "Is the impact local, global, or systemic?"
1. "Is the return on investment of the possible solution greater than the cost of maintenance?"
1. "Will this block our ability to deliver new features or to deliver faster?"

Maintenance Issues Triage

Engineers are responsible for the triage of maintenance issues using the scheme above and applying the labels below. From time to time the engineering manager may also triage maintenance issues with validation and support from engineers. All maintenance issues should have one of the following scoped labels.

- urgent-important will translate to priority::1
- not-urgent-important will translate to priority::2
- urgent-not-important will translate to priority::3
- not-urgent-not-important will translate to priority::4

Adding Maintenance Issues to the Current Milestone

For each milestone, we should allocate (at a minimum) maintenance issues equal to the number of engineers per discipline. These should be chosen based on the prioritization scheme outlined above.

Labels and How We Use Them

We have many labels that can be applied to an issue or a merge request. Besides the issue workflow labels, here are the minimum basic labels to apply to issues and merge requests:

- Type (type::feature, type::bug, or type::maintenance)
- Stage that owns the area (section::fulfillment)
- Group that owns the area (group::utilization)
- Discipline (frontend, backend, database, UX, documentation)
- security if the issue is related to application security, and breaking change if this work is considered a breaking change

The extended list of labels that should be applied where applicable to issues and MRs:

- UI Context: usagequota:storage, usagequota:pipeline, usagequota:transfer, usagequota:other

The workflow::planning breakdown label is driven by Product, but is a collaborative effort between Product, UX and Engineering. We are using this label to help drive the refinement, estimation, and breakdown of issues as described above. During that refinement process, the following examples of how the team might contribute towards that effort:

- Product defining or clarifying the problem statement. Product and UX will collaborate on problem validation as needed.
- UX providing designs as needed.
- UX reviewing issues that have a workflow/user experience change by monitoring issues with the ~UX label.
- Engineering clarifying the issue description as stated, and refines and weights the issue once Product and UX have provided enough details to do so.

Spikes are labeled with spike, feature::enhancement and type::feature. Spikes that update dependencies or upgrade versions of underlying libraries are labeled with spike, maintenance::dependency and type::maintenance

Estimation

We follow the estimation rubric provided on the Fulfillment handbook page that equates issue weight mostly with complexity and breadth of change required.

As a general rule, issues with an estimated weight above 5 should be broken down into smaller, more iterative issues. The Engineer assigned to the issue for estimation will suggest how to split the issue. If the Engineer has a clear understanding about how the split should happen, they should proactively split the issue while also leaving estimations on the child issues along with a comment on the parent issue. If the split results is more than 4 issues, consider creating an epic.

Estimations should help us:

- better understand the goal of the issue
- understand what a solution could be, but not necessarily all of the implementation details
- assess whether blockers exist or not

Estimations are not perfect, nor should we spend many hours trying to provide the most perfect estimation. As a result, we do not expect:

- implementation details to be heavily scrutinized

- all blockers to be caught
- that we prove the proposed solution to be the right one

Sometimes during development, we reshape the issue, change its solution, or even postpone it. We are proud to work to contribute to issues even if the outcome shifts from our original notion.

Estimation Review

We do not have a synchronous meeting to discuss issue estimates like in a planning poker session where estimations are blindly chosen and shared only to reach a consensus from the entire group; this activity does make estimations more engaging and accurate than other methods. Like other GitLab activities, we attempt to do this asynchronously, but limited to a few team members.

In an effort to raise our confidence of our estimates through shared discussion, ask another team member to briefly review the issue and your rationale for estimation, but conceal the weight of your estimate in a details block (see below). Should both team members agree, or be moved to either side, we will use that estimate. If there is some disagreement, about the estimate value provided as a result of new information, experience, or opinion, we have options to choose either value, an average value that fits within the estimation framework, or seeking a third party opinion like that of the engineering manager.

```
html
<details>
  <summary>Estimate</summary>
  estimate: 2
</details>
```

Throughout the estimation process, consider the following GitLab sub-values:

- Bias for action: the person with more context should help drive the estimation
- Everything is in draft: other team members can propose changes to the estimate with an explanation
- Low level of shame: estimations will never be perfect, but we can quickly iterate on an estimate that will be good enough to get us started
- Articulate when you've changed your mind: as we arrive towards some consensus estimate, clearly articulate that your earlier stance is no longer your current stance

Spikes

We use spikes during the milestone to produce a design document or similar artifact that is useful for planning and execution of feature or other complex work. This can simply be, but not limited to, an issue containing a summary of the discussions on the topic, answers to questions from the spike description, links to any Proof of Concept MRs produced, a road map or other detailed outline. They are assigned like other work and take up capacity from the milestone.

Spikes are typically considered a Deliverable issue and we use this template (internal only) to create new spike issues.

Estimating the effort required for a Spike is not as clearly set or easily defined as feature work because the complexity can't be accurately estimated. Take into consideration the following criteria when adding a weight to spike issue.

- Scope
- Duration (time box)
- Outcome / Deliverable / Artifact production

Follow the same Fibonacci scale used for feature work from 1 (low, quick, easy) to 5 (large, lengthy, difficult). Historically, we have not estimated spikes higher than 5.

Guidelines

- Spikes are typically marked as deliverable to ensure clear ownership from engineers.
- Spikes are typically time-boxed to a short duration, sometimes one week, never more than one milestone.
- We should try to schedule Spikes at the beginning of a milestone so that the investigation will conclude before the planning of the following milestone.
- Limit: no more than 2 Spikes per milestone.
- Typically multiple team members collaborate on a spike. We want to ensure we get multiple different viewpoints as well as keep the research focused and efficient.
- Spikes should be assigned to at least one frontend and one backend engineer to ensure we consider the issue from both viewpoints. Otherwise, we can split the spike issue along those disciplines.

"What do I work on next?"

What are the priorities for this milestone?

- When we receive P1/S1 bug type issues, they will outrank other planned issues.
- The highest priorities for a given milestone would be issues labeled with Deliverable and group::utilization.
- Once all Deliverable issues have been picked up as denoted by assignment and are in the workflow:in dev stage or beyond, then you should pull from the current milestone column on the respective boards. The current milestone column will be stack ranked (or at least reviewed) daily by the Product Manager, so that each team member can pull from the top of the column expecting that it is already ordered in priority.
- Complete assigned estimation work.
- Refine the issue backlog.
- Stuff that should just work[^stsjw-1], unscheduled work, or other areas of interest.

DRIs assign themselves to the selected issue. This is also a good time to re-evaluate the weight and proposal, in case the DRI picking up the issue was not the same individual who originally weighted and refined the issue. Aspirationally, we strive to iterate and want to breakdown issues to ship as much value in the milestone for our users as possible, which means if you see a more efficient way forward when you start working on a new issue, you are free to raise a comment and update the proposal to deliver more iterative value.

[^stsjw-1]: Small effort improvements to the project can be added to the current milestone as unplanned work. Resolving small bugs or fixing flakey tests immediately are examples of how to

use the stuff that should just work label.

Unblocking Frontend work

Given the higher capacity to work on Frontend issues, we can use the following system for Frontend Engineers to ping Backend engineers in their timezone to resolve the blockers or backend dependencies faster:

Depending on the priority of the project or OKR, Frontend engineers are able to ask Backend engineers for help regardless of their current project - unless the priority of the Backend work is higher. In the above table, the cells in green show the times when Frontend Engineers can ping the Backend engineers for help regardless of their DRI status of the project, while cells in yellow may require an escalation to the EM if they are blocked or to wait for the Backend DRI of the project to help out.

Triage

The following lists are links to Sentry and other tools where we proactively identify and triage Utilization problems. Proactive triage will not only provide for a more secure and robust application, but also provide for a better user experience especially as we iterate on features, reveal features from behind a feature flag, or introduce a refactoring. It leans into our Bias for action sub-value and raises our awareness of application performance.

Potential list of places to check

Subject	Link	Notes
-	-	-
CustomersDot syncing	Sentry	UpdateGitlabPlanInfoWorker class is used to sync data between CustomersDot and GitLab
GitLab Subscriptions	Sentry	Results could be refined by controller, e.g. SubscriptionsController
Billing errors	Sentry	Results could be further refined by controller, e.g. Groups::BillingsController, Projects::BillingsController
Rails logs	Kibana	Utilization feature category Rails logs for the last 7 days
Sidekiq logs	Kibana	Utilization feature category Sidekiq logs for the last 7 days
Billable Member API	Grafana dashboard	-
CustomersDot Bug Issues	Issues	-
GitLab Bug Issues	Issues	-

Creating an issue direct from Sentry

There's a way in sentry to create an issue for any error you find.

e.g. <https://sentry.gitlab.net/gitlab/customersgitlabcom/issues/2505559/?query=is%3Aunresolved%20UpdateGitlabPlanInfoWorker>

See links in the right sidebar:

Although both links